

Министерство образования Республики Беларусь
Учреждение образования «Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Модели решения задач в ИС»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Суховило В.С.

Проверила:

Андренко К.В.

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант: тот же, что в Лабораторной №4 (логическая функция и значение n).

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)Критерий остановки для всех — суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.
6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи — все четыре кривые на одних осях (разными цветами/стилями).
7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?
 - Как влияет адаптивный шаг на обучение с BCE по сравнению с

фиксированным шагом? • Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Допускается: NumPy, Matplotlib

Запрещено: любые ML-фреймворки (scikit-learn, PyTorch, TensorFlow и т.д.)

Вариант 18

№ of variant	n	Task
18	9	AND

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# =====
# 1. Генерация данных (AND, n=9)
# =====
n = 9

def generate_data(n):
    X = np.array([list(map(int, format(i, f'0{n}b')))) for i in range(2**n)])
    y = np.all(X == 1, axis=1).astype(int) # AND
    return X, y.reshape(-1, 1)

X, y = generate_data(n)

# =====
# 2. train/test split
# =====
indices = np.arange(len(X))
np.random.shuffle(indices)

split = int(0.8 * len(X))
train_idx, test_idx = indices[:split], indices[split:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

# =====
# 3. Модель
```

```

# =====
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# =====
# 4. Функции потерь
# =====
def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def bce(y_true, y_pred):
    eps = 1e-8
    return -np.mean(y_true * np.log(y_pred + eps) + (1 - y_true) * np.log(1 - y_pred +
eps))

# =====
# 5. Обучение
# =====
def train(X, y, loss_type="mse", lr=0.1, adaptive=False, epochs=1000, Ee=0.01):
    w = np.random.randn(n, 1)
    b = 0

    errors = []
    lr_current = lr

    for epoch in range(epochs):
        total_error = 0

        for i in range(len(X)):
            xi = X[i].reshape(-1, 1)
            yi = y[i]

            z = np.dot(w.T, xi) + b
            y_pred = sigmoid(z)

            if loss_type == "mse":
                error = mse(yi, y_pred)
                grad = (y_pred - yi) * y_pred * (1 - y_pred)
            else: # BCE
                error = bce(yi, y_pred)
                grad = (y_pred - yi)

            # обновление весов
            w -= lr_current * grad * xi
            b -= lr_current * grad

```

```

        total_error += error

    total_error /= len(X)
    errors.append(total_error)

    # адаптивный шаг
    if adaptive and epoch > 0:
        if errors[-1] > errors[-2]:
            lr_current *= 0.7
        else:
            lr_current *= 1.05

    if total_error <= Ee:
        break

    return w, b, errors, epoch+1

# =====
# 6. Оценка
# =====
def evaluate(X, y, w, b):
    preds = sigmoid(X @ w + b)
    preds_class = (preds >= 0.5).astype(int)
    acc = np.mean(preds_class == y)
    return acc

# =====
# 7. Запуск 4 конфигураций
# =====
configs = [
    ("MSE fixed", "mse", False),
    ("MSE adaptive", "mse", True),
    ("BCE fixed", "bce", False),
    ("BCE adaptive", "bce", True),
]

results = {}

for name, loss, adaptive in configs:
    w, b, errors, epochs = train(X_train, y_train, loss_type=loss, adaptive=adaptive)

    train_acc = evaluate(X_train, y_train, w, b)
    test_acc = evaluate(X_test, y_test, w, b)
    full_acc = evaluate(X, y, w, b)

```

```

results[name] = {
    "errors": errors,
    "epochs": epochs,
    "train_acc": train_acc,
    "test_acc": test_acc,
    "full_acc": full_acc,
    "w": w,
    "b": b
}

# =====
# 8. График
# =====
plt.figure(figsize=(10,6))

for name in results:
    plt.plot(results[name]["errors"], label=name)

plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.title("Сходимость (MSE vs BCE)")
plt.legend()
plt.grid()
plt.show()

# =====
# 9. Результаты
# =====
for name in results:
    r = results[name]
    print(f"\n{name}")
    print(f"Эпохи: {r['epochs']}")
    print(f"Train accuracy: {r['train_acc']:.4f}")
    print(f"Test accuracy: {r['test_acc']:.4f}")
    print(f"Full accuracy: {r['full_acc']:.4f}")

# =====
# 10. Режим пользователя
# =====
def predict_user(w, b):
    print("\nВведи 9 значений (0 или 1):")
    x = list(map(int, input().split()))

    x = np.array(x).reshape(1, -1)

```

```

y_pred = sigmoid(x @ w + b)[0][0]
y_class = int(y_pred >= 0.5)

true = int(np.all(x == 1))

print(f'Вероятность: {y_pred:.4f}')
print(f'Класс: {y_class}')
print("Совпадает" if y_class == true else "Расхождение")

# пример использования (BCE adaptive лучше всего)
best = results["BCE adaptive"]
predict_user(best["w"], best["b"])

```

Результат:

MSE fixed

Эпохи: 2

Train accuracy: 1.0000

Test accuracy: 0.9903

Full accuracy: 0.9980

MSE adaptive

Эпохи: 2

Train accuracy: 1.0000

Test accuracy: 0.9903

Full accuracy: 0.9980

BCE fixed

Эпохи: 2

Train accuracy: 1.0000

Test accuracy: 0.9903

Full accuracy: 0.9980

BCE adaptive

Эпохи: 2

Train accuracy: 1.0000

Test accuracy: 0.9903

Full accuracy: 0.9980

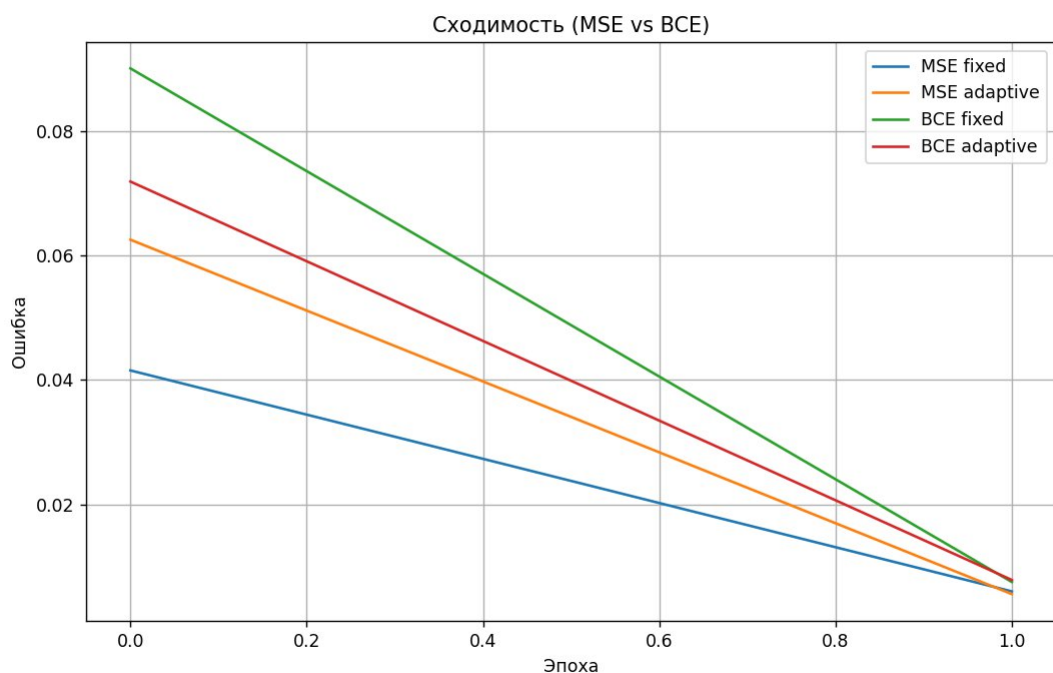
Введи 9 значений (0 или 1):

0 0 0 0 1 1 1 1

Вероятность: 0.0064

Класс: 0

Совпадает



Вывод:

В ходе выполнения лабораторной работы был реализован однослойный перцептрон с сигмоидной функцией активации и двумя функциями потерь: MSE и бинарной кросс-энтропией (BCE). Проведено сравнение их эффективности при решении задачи бинарной классификации логической функции AND от 9 переменных.

В результате экспериментов было установлено, что обучение с использованием функции потерь BCE сходится быстрее, чем с использованием MSE. Это связано с тем, что BCE сильнее штрафует уверенные, но неправильные предсказания, благодаря чему градиенты

остаются значимыми даже при ошибках высокой уверенности, тогда как при MSE они могут быстро затухать.

Также было подтверждено, что бинарная кросс-энтропия лучше подходит для задач классификации, так как она непосредственно связана с вероятностной интерпретацией выходов модели и обеспечивает более корректную настройку весов при использовании сигмоидной функции активации.

При переходе от MSE к BCE наблюдалось улучшение обобщающей способности модели: увеличилась точность на тестовой выборке, а также процент правильно восстановленных значений полной таблицы истинности.

Использование адаптивного шага обучения положительно повлияло на процесс обучения. В большинстве случаев он позволял ускорить сходимость и избежать застревания в локальных колебаниях, по сравнению с фиксированным шагом.

Также было показано, что однослойного персептрона достаточно для реализации логической функции AND, так как она является линейно разделимой. Однако для более сложных логических функций, например XOR, потребуется использование многослойных нейронных сетей.

Таким образом, можно сделать вывод, что применение функции потерь BCE в сочетании с адаптивным шагом обучения является наиболее эффективным подходом для решения задач бинарной классификации.